

Rust syntax

The **syntax of Rust** is the set of rules defining how a Rust program is written and compiled.

Rust's syntax is similar to that of C and C++,^{[1][2]} although many of its features were influenced by functional programming languages such as OCaml.^[3]



A snippet of Rust code



Basics

Although Rust syntax is heavily influenced by the syntaxes of C and C++, the syntax of Rust is far more distinct from C++ syntax than Java or C#, as those languages have more C-style declarations, primitive names, and keywords.

Below is a "Hello, World!" program in Rust. The `fn` keyword denotes a function, and the `println!` macro (see § Macros) prints the message to standard output.^[4] Statements in Rust are separated by semicolons.

```
fn main() {  
    println!("Hello, World!");  
}
```

Reserved words

Keywords

The following 42 words are reserved, and may not be used as identifiers.

- | | |
|-------------------------|----------------------|
| ▪ <code>as</code> | ▪ <code>fn</code> |
| ▪ <code>async</code> | ▪ <code>for</code> |
| ▪ <code>await</code> | ▪ <code>if</code> |
| ▪ <code>break</code> | ▪ <code>impl</code> |
| ▪ <code>const</code> | ▪ <code>in</code> |
| ▪ <code>continue</code> | ▪ <code>let</code> |
| ▪ <code>crate</code> | ▪ <code>loop</code> |
| ▪ <code>dyn</code> | ▪ <code>match</code> |
| ▪ <code>else</code> | ▪ <code>mod</code> |
| ▪ <code>enum</code> | ▪ <code>move</code> |
| ▪ <code>extern</code> | ▪ <code>mut</code> |
| ▪ <code>false</code> | ▪ <code>pub</code> |

- ref
- return
- Self
- self
- static
- struct
- super
- trait
- true
- type
- union
- unsafe
- use
- where
- while

Unused words

The following words are reserved as keywords, but currently have no use or purpose. There are 14 unused words.

- abstract
- become
- box
- do
- final
- gen
- macro
- override
- priv
- try
- typeof
- unsized
- virtual
- yield

Variables

Variables in Rust are defined through the `let` keyword.^[5] The example below assigns a value to the variable with name `foo` and outputs its value.

```
fn main() {
    let foo: i32 = 10;
    println!("The value of foo is {foo}");
}
```

Variables are immutable by default, but adding the `mut` keyword allows the variable to be mutated.^[6] The following example uses `//`, which denotes the start of a comment.^[7]

```
fn main() {
    // This code would not compile without adding "mut".
    let mut foo: i32 = 10;
    println!("The value of foo is {foo}");
    foo = 20;
    println!("The value of foo is {foo}");
}
```

Multiple `let` expressions can define multiple variables with the same name, known as variable shadowing. Variable shadowing allows transforming variables without having to name the variables differently.^[8] The example below declares a new variable with the same name that is double the original value:

```
fn main() {
    let foo: i32 = 10;
    // This will output "The value of foo is 10"
    println!("The value of foo is {foo}");
    let foo: i32 = foo * 2;
    // This will output "The value of foo is 20"
    println!("The value of foo is {foo}");
}
```

Variable shadowing is also possible for values of different types. For example, going from a string to its length in bytes:

```
fn main() {
    let letters: str = "abc";
    let letters: usize = letters.len();
}
```

Block expressions and control flow

A *block expression* is delimited by curly brackets. When the last expression inside a block does not end with a semicolon, the block evaluates to the value of that trailing expression:^[9]

```
fn main() {
    let x: i32 = {
        println!("this is inside the block");
        1 + 2
    };
    println!("1 + 2 = {x}");
}
```

Trailing expressions of function bodies are used as the return value:^[10]

```
fn add_two(x: i32) -> i32 {
    x + 2
}
```

if expressions

An *if conditional expression* executes code based on whether the given value is `true`. `else` can be used for when the value evaluates to `false`, and `else if` can be used for combining multiple expressions.^[11]

```
fn main() {
    let x: i32 = 10;
    if x > 5 {
        println!("value is greater than five");
    }

    if x % 7 == 0 {
        println!("value is divisible by 7");
    } else if x % 5 == 0 {
        println!("value is divisible by 5");
    } else {
        println!("value is not divisible by 7 or 5");
    }
}
```

`if` and `else` blocks can evaluate to a value, which can then be assigned to a variable:^[11]

```
fn main() {  
    let x: i32 = 10;  
    let new_x: i32 = if x % 2 == 0 { x / 2 } else { 3 * x + 1 };  
    println!("{}", new_x);  
}
```

while loops

`while` can be used to repeat a block of code while a condition is met.^[12]

```
fn main() {  
    // Iterate over all integers from 4 to 10  
    let mut value: i32 = 4;  
    while value <= 10 {  
        println!("value = {value}");  
        value += 1;  
    }  
}
```

for loops and iterators

For loops in Rust loop over elements of a collection.^[13] `for` expressions work over any `Iterator` type.

```
fn main() {  
    // Using `for` with range syntax for the same functionality as above  
    // The syntax 4..=10 means the range from 4 to 10, up to and including 10.  
    for value in 4..=10 {  
        println!("value = {value}");  
    }  
}
```

In the above code, `4..=10` is a value of type `Range` which implements the `Iterator` trait. The code within the curly braces is applied to each element returned by the iterator.

Iterators can be combined with functions over iterators like `map`, `filter`, and `sum`. For example, the following adds up all numbers between 1 and 100 that are multiples of 3:

```
(1..=100).filter(|&x: i8| -> bool x % 3 == 0).sum()
```

loop and break statements

More generally, the `loop` keyword allows repeating a portion of code until a `break` occurs. `break` may optionally exit the loop with a value. In the case of nested loops, labels denoted by `'label_name'` can be used to break an outer loop rather than the innermost loop.^[14]

```
fn main() {  
    let value: i32 = 456;  
    let mut x: i32 = 1;  
    let y = loop {  
        x *= 10;  
        if x > value {  
            break x / 10;  
        }  
    };  
};
```

```
println!("largest power of ten that is smaller than or equal to value: {y}");

let mut up: i32 = 1;
'outer: loop {
    let mut down: i32 = 120;
    loop {
        if up > 100 {
            break 'outer;
        }

        if down < 4 {
            break;
        }

        down /= 2;
        up += 1;
        println!("up: {up}, down: {down}");
    }
    up *= 2;
}
}
```

Pattern matching

The `match` and `if let` expressions can be used for pattern matching. For example, `match` can be used to double an optional integer value if present, and return zero otherwise:^[15]

```
fn double(x: Option<u64>) -> u64 {
    match x {
        Some(y) => y * 2,
        None => 0,
    }
}
```

Equivalently, this can be written with `if let` and `else`:

```
fn double(x: Option<u64>) -> u64 {
    if let Some(y) = x {
        y * 2
    } else {
        0
    }
}
```

Types

Rust is strongly typed and statically typed, meaning that the types of all variables must be known at compilation time. Assigning a value of a particular type to a differently typed variable causes a compilation error. Type inference is used to determine the type of variables if unspecified.^[16]

The type `()`, called the "unit type" in Rust, is a concrete type that has exactly one value (itself). It occupies no memory (as it represents the absence of value). All functions that do not have an indicated return type implicitly return `()`. It is similar to `void` in other C-style languages, however `void` denotes the absence of a type and cannot have any value.

The default integer type is `i32`, and the default floating point type is `f64`. If the type of a literal number is not explicitly provided, it is either inferred from the context or the default type is used.^[17]

Primitive types

Integer types in Rust are named based on the signedness and the number of bits the type takes. For example, `i32` is a signed integer that takes 32 bits of storage, whereas `u8` is unsigned and only takes 8 bits of storage. `isize` and `usize` take storage depending on the architecture of the computer that runs the code, for example, on computers with 32-bit architectures, both types will take up 32 bits of space.

By default, integer literals are in base-10, but different radices are supported with prefixes, for example, `0b11` for binary numbers, `0o567` for octals, and `0xDB` for hexadecimals. By default, integer literals default to `i32` as its type. Suffixes such as `4u32` can be used to explicitly set the type of a literal.^[18] Byte literals such as `b'X'` are available to represent the ASCII value (as a `u8`) of a specific character.^[19]

The Boolean type is referred to as `bool` which can take a value of either `true` or `false`. A `char` takes up 32 bits of space and represents a Unicode scalar value:^[20] a Unicode codepoint that is not a surrogate.^[21] IEEE 754 floating point numbers are supported with `f32` for single precision floats and `f64` for double precision floats.^[22]

Compound types

Compound types can contain multiple values. Tuples are fixed-size lists that can contain values whose types can be different. Arrays are fixed-size lists whose values are of the same type. Expressions of the tuple and array types can be written through listing the values, and can be accessed with `.index` or `[index]`.^[23]

```
let tuple: (u32, i64) = (3, -3);
let array: [i8; 5] = [1, 2, 3, 4, 5];
let value: i64 = tuple.1; // -3
let value: i8 = array[2]; // 3
```

Arrays can also be constructed through copying a single value a number of times:^[24]

```
let array2: [char; 10] = [' ', 10];
```

Ownership and references

Rust's ownership system consists of rules that ensure memory safety without using a garbage collector. At compile time, each value must be attached to a variable called the *owner* of that value, and every value must have exactly one owner.^[25] Values are moved between different owners through assignment or passing a value as a function parameter. Values can also be *borrowed*, meaning they are temporarily passed to a different function before being returned to the owner.^[26] With these rules, Rust can prevent the creation and use of dangling pointers.^{[26][27]}

```
fn print_string(s: String) {
    println!("{}", s);
}

fn main() {
```

```

let s: String = String::from("Hello, World");
print_string(s); // s consumed by print_string
// s has been moved, so cannot be used any more
// another print_string(s); would result in a compile error
}

```

The function `print_string` takes ownership over the `String` value passed in; Alternatively, `&` can be used to indicate a reference type (in `&String`) and to create a reference (in `&s`):^[28]

```

fn print_string(s: &String) {
    println!("{}", s);
}

fn main() {
    let s: String = String::from("Hello, World");
    print_string(&s); // s borrowed by print_string
    print_string(&s); // s has not been consumed; we can call the function many times
}

```

Because of these ownership rules, Rust types are known as linear or affine types, meaning each value can be used exactly once. This enforces a form of software fault isolation as the owner of a value is solely responsible for its correctness and deallocation.^[29]

When a value goes out of scope, it is *dropped* by running its destructor. The destructor may be programmatically defined through implementing the `Drop` trait. This helps manage resources such as file handles, network sockets, and locks, since when objects are dropped, the resources associated with them are closed or released automatically.^[30]

Lifetimes

Object lifetime refers to the period of time during which a reference is valid; that is, the time between the object creation and destruction.^[31] These *lifetimes* are implicitly associated with all Rust reference types. While often inferred, they can also be indicated explicitly with named lifetime parameters (often denoted `'a`, `'b`, and so on).^[32]

Lifetimes in Rust can be thought of as lexically scoped, meaning that the duration of an object lifetime is inferred from the set of locations in the source code (i.e., function, line, and column numbers) for which a variable is valid.^[33] For example, a reference to a local variable has a lifetime corresponding to the block it is defined in.^[33]

```

fn main() {
    let x: i32 = 5;           // -----+- Lifetime 'a
                             //          |
    let r: &i32 = &x;         // -+-- Lifetime 'b  |
                             //      |           |
    println!("r: {}", r);    //      |           |
                             //      |           |
                             //      |           |
                             // -+          |
                             // -----+-
}

```

The borrow checker in the Rust compiler then enforces that references are only used in the locations of the source code where the associated lifetime is valid.^{[34][35]} In the example above, storing a reference to variable `x` in `r` is valid, as variable `x` has a longer lifetime (`'a`) than variable `r` (`'b`). However, when `x` has a shorter lifetime, the borrow checker would reject the program:

```

fn main() {
  let r: &i32;           // -----+ Lifetime 'a
                        //          |
  {
    let x: i32 = 5;      // -+-- Lifetime 'b |
    r = &x; // ERROR: x does |
  }                     // not live long -|
                        // enough         |
  println!("r: {}", r); //          |
}                       // -----+

```

Since the lifetime of the referenced variable ('b) is shorter than the lifetime of the variable holding the reference ('a), the borrow checker errors, preventing x from being used from outside its scope.^[36]

Lifetimes can be indicated using explicit *lifetime parameters* on function arguments. For example, the following code specifies that the reference returned by the function has the same lifetime as `original` (and *not* necessarily the same lifetime as `prefix`):^[37]

```

fn remove_prefix<'a>(mut original: &'a str, prefix: &str) -> &'a str {
  if original.starts_with(prefix) {
    original = original[prefix.len()..];
  }
  original
}

```

In the compiler, ownership and lifetimes work together to prevent memory safety issues such as dangling pointers.^{[38][39]}

User-defined types

User-defined types are created with the `struct` or `enum` keywords. The `struct` keyword is used to denote a record type that groups multiple related values.^[40] `enums` can take on different variants at runtime, with its capabilities similar to algebraic data types found in functional programming languages.^[41] Both records and enum variants can contain fields with different types.^[42] Alternative names, or aliases, for the same type can be defined with the `type` keyword.^[43]

The `impl` keyword can define methods for a user-defined type. Data and functions are defined separately. Implementations fulfill a role similar to that of classes within other languages.^[44]

Standard library

Summary of Rust's types in the standard library

Type	Description	Examples
String	UTF-8-encoded strings (dynamic)	String::new() String::from("Hello") "🦀🦀🦀".to_string()
OsStr OsString	Platform-native strings ^[note 1] (borrowed ^[45] and dynamic ^[46])	OsStr::new("Hello") OsString::from("world")
Path PathBuf	Paths (borrowed ^[47] and dynamic ^[48])	Path::new("./path/to") PathBuf::from(r"C:\path\to")
CStr CString	C-compatible, null-terminated strings (borrowed ^[49] and dynamic ^[49])	c"Hello" CStr::from_bytes_with_nul(b"Hello\0").unwrap() CString::new("world").unwrap()
Vec<T>	<u>Dynamic arrays</u>	Vec::new() vec![1, 2, 3, 4, 5]
Option<T>	<u>Option type</u>	None Some(3) Some("hello")
Result<T, E>	<u>Error handling using a result type</u>	Ok(3) Err("something went wrong")
Box<T>	A pointer to a heap-allocated value. ^[49] Similar to C++'s <code>std::unique_ptr</code> (https://en.cppreference.com/w/cpp/memory/unique_ptr).	<pre>let boxed: Box<u8> = Box::new(5); let val: u8 = *boxed;</pre>
Rc<T>	<u>Reference counting pointer</u> ^[50]	<pre>let five: Rc<u8> = Rc::new(5); let also_five: Rc<u8> = five.clone();</pre>
Arc<T>	<u>Atomic, thread-safe reference counting pointer</u> ^[51]	<pre>let foo: Arc<Vec<f32>> = Arc::new(vec![1.0, 2.0]); let a: Arc<Vec<f32>> = foo.clone(); // a can be sent to another thread</pre>
Cell<T>	A mutable memory location ^[52]	<pre>let c: Cell<u8> = Cell::new(5); c.set(10);</pre>

Mutex<T>	A mutex lock for shared data contained within. ^[53]	<pre>let mutex: Mutex<u32> = Mutex::new(0_u32); let _guard: LockResult<MutexGuard<'_, u32>> = mutex.lock();</pre>
RwLock<T>	Readers–writer lock ^[54]	<pre>let lock: RwLock<u8> = RwLock::new(5); let r1: u8 = lock.read().unwrap();</pre>
Condvar	A conditional monitor for shared data ^[55]	<pre>let (lock, cvar): (Mutex<bool>, Condvar) = (Mutex::new(true), Condvar::new()); // As long as the value inside the `Mutex<bool>` is `true`, we wait. let _guard: Mutex<bool> = cvar.wait_while(lock.lock().unwrap(), pending { *pending }).unwrap();</pre>
Duration	Type that represents a span of time ^[56]	<pre>Duration::from_millis(1) // 1ms</pre>
HashMap<K, V>	Hash table ^[57]	<pre>let mut player_stats: HashMap<String, u32> = HashMap::new(); player_stats.insert("damage", 1); player_stats.entry("health").or_insert(100);</pre>
BTreeMap<K, V>	B-tree ^[58]	<pre>let mut solar_distance: BTreeMap<String, f32> = BTreeMap::from([("Mercury", 0.4), ("Venus", 0.7),]); solar_distance.entry("Earth").or_insert(1.0);</pre>

Option values are handled using syntactic sugar, such as the `if let` construction, to access the inner value (in this case, a string):^[59]

```
fn main() {
    let name1: Option<&str> = None;
    // In this case, nothing will be printed out
    if let Some(name) = name1 {
        println!("{}", name);
    }

    let name2: Option<&str> = Some("Matthew");
    // In this case, the word "Matthew" will be printed out
    if let Some(name) = name2 {
        println!("{}", name);
    }
}
```

Pointers

Summary of Rust's pointer and reference primitive types

Type	Description	Examples
<code>&T</code> <code>&mut T</code>	<u>References</u> (immutable and mutable)	<pre>let x_ref: &T = &x; let x_ref: &mut T = &mut x;</pre>
<code>Option<&T></code> <code>Option<&mut T></code>	Option wrapped reference Possibly null reference	<pre>None let x_ref: Option<&T> = Some(&x); let x_ref: Option<&mut T> = Some(&mut x);</pre>
<code>Box<T></code> <code>Option<Box<T>></code>	A pointer to heap-allocated value (or possibly null pointer if wrapped in option) ^[49]	<pre>let boxed: Box<u8> = Box::new(0); let boxed: Option<Box<String>> = Some(Box::new("Hello World"));</pre>
<code>*const T</code> <code>*mut T</code>	Raw pointers (immutable and mutable) Possibly <u>null</u> ; unsafe to <u>dereference</u>	<pre>let x_ptr: *const T = &x as *const T; let x_ptr: *mut T = &mut x as *mut T;</pre>

To prevent the use of null pointers and their dereferencing, the basic `&` and `&mut` references are guaranteed to not be null. Rust instead uses `Option` for this purpose: `Some(T)` indicates that a value is present, and `None` is analogous to the null pointer.^[60] `Option` implements a "null pointer optimization", avoiding any spatial overhead for types that cannot have a null value (references or the `NonZero` types, for example).^[61] Though null pointers are idiomatically avoided, the null pointer constant in Rust is represented by `std::ptr::null()`.

Rust also supports raw pointer types `*const` and `*mut`, which may be null; however, it is impossible to dereference them unless the code is explicitly declared `unsafe` through the use of an `unsafe` block. Unlike dereferencing, the creation of raw pointers is allowed inside of safe Rust code.^[62]

Type conversion

Rust provides no implicit type conversion (coercion) between most primitive types. But, explicit type conversion (casting) can be performed using the `as` keyword.^[63]

```
let x: i32 = 1000;  
println!("1000 as a u16 is: {}", x as u16);
```

Polymorphism

Generics

Rust's more advanced features include the use of generic functions. A generic function is given generic parameters, which allow the same function to be applied to different variable types. This capability reduces duplicate code^[64] and is known as parametric polymorphism.

The following program calculates the sum of two things, for which addition is implemented using a generic function:

```
use std::ops::Add;

// sum is a generic function with one type parameter, T
fn sum<T>(num1: T, num2: T) -> T
where
    T: Add<Output = T>, // T must implement the Add trait where addition returns another T
{
    num1 + num2 // num1 + num2 is syntactic sugar for num1.add(num2) provided by the Add trait
}

fn main() {
    let result1: i32 = sum(10, 20);
    println!("Sum is: {}", result1); // Sum is: 30

    let result2: f32 = sum(10.23, 20.45);
    println!("Sum is: {}", result2); // Sum is: 30.68
}
```

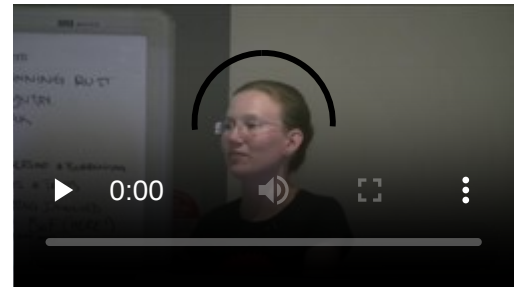
At compile time, polymorphic functions like `sum` are instantiated with the specific types the code requires; in this case, sum of integers and sum of floats.

Generics can be used in functions to allow implementing a behavior for different types without repeating the same code. Generic functions can be written in relation to other generics, without knowing the actual type.^[65]

Traits

Rust's type system supports a mechanism called traits, inspired by type classes in the Haskell language,^[66] to define shared behavior between different types. For example, the `Add` trait can be implemented for floats and integers, which can be added; and the `Display` or `Debug` traits can be implemented for any type that can be converted to a string. Traits can be used to provide a set of common behavior for different types without knowing the actual type. This facility is known as ad hoc polymorphism.

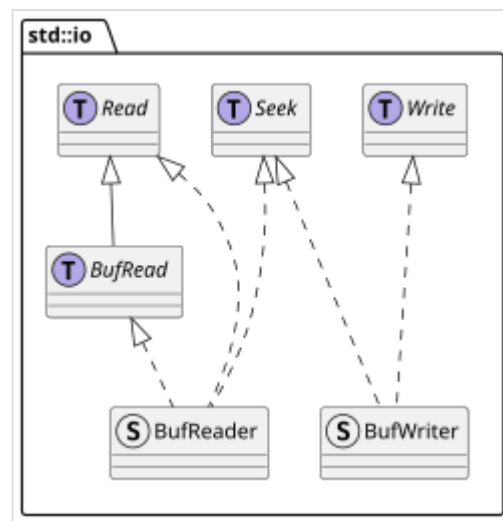
Generic functions can constrain the generic type to implement a particular trait or traits; for example, an `add_one` function might require the type to implement `Add`. This means that a generic function can be type-checked as soon as it is defined. The implementation of generics is similar to the typical implementation of C++ templates: a separate copy of the code is generated for each instantiation. This is



A presentation on Rust by Emily Dunham from [Mozilla's Rust team](#) ([linux.conf.au](#) conference, Hobart, 2017)

called monomorphization and contrasts with the type erasure scheme typically used in Java and Haskell. Type erasure is also available via the keyword `dyn` (short for dynamic).^[67] Because monomorphization duplicates the code for each type used, it can result in more optimized code for specific-use cases, but compile time and size of the output binary are also increased.^[68]

In addition to defining methods for a user-defined type, the `impl` keyword can be used to implement a trait for a type.^[44] Traits can provide additional derived methods when implemented.^[69] For example, the trait `Iterator` requires that the `next` method be defined for the type. Once the `next` method is defined, the trait can provide common functional helper methods over the iterator, such as `map` or `filter`.^[70]



Excerpt from `std::io`

Trait objects

Rust traits are implemented using static dispatch, meaning that the type of all values is known at compile time; however, Rust also uses a feature known as *trait objects* to accomplish dynamic dispatch, a type of polymorphism where the implementation of a polymorphic operation is chosen at runtime. This allows for behavior similar to duck typing, where all data types that implement a given trait can be treated as functionally equivalent.^[71] Trait objects are declared using the syntax `dyn Tr` where `Tr` is a trait. Trait objects are dynamically sized, therefore they must be put behind a pointer, such as `Box`.^[72] The following example creates a list of objects where each object can be printed out using the `Display` trait:

```

use std::fmt::Display;

let v: Vec<Box<dyn Display>> = vec![
    Box::new(3),
    Box::new(5.0),
    Box::new("hi"),
];

for x in v {
    println!("{}", x);
}

```

If an element in the list does not implement the `Display` trait, it will cause a compile-time error.^[73]

Memory safety

Rust is designed to be memory safe. It does not permit null pointers, dangling pointers, or data races.^{[74][75][76][77]} Data values can be initialized only through a fixed set of forms, all of which require their inputs to be already initialized.^[78]

Unsafe code can subvert some of these restrictions, using the `unsafe` keyword.^[62] Unsafe code may also be used for low-level functionality, such as volatile memory access, architecture-specific intrinsics, type punning, and inline assembly.^[79]

Memory management

Rust does not use garbage collection. Memory and other resources are instead managed through the "resource acquisition is initialization" convention,^[80] with optional reference counting. Rust provides deterministic management of resources, with very low overhead.^[81] Values are allocated on the stack by default, and all dynamic allocations must be explicit.^[82]

The built-in reference types using the `&` symbol do not involve run-time reference counting. The safety and validity of the underlying pointers is verified at compile time, preventing dangling pointers and other forms of undefined behavior.^[83] Rust's type system separates shared, immutable references of the form `&T` from unique, mutable references of the form `&mut T`. A mutable reference can be coerced to an immutable reference, but not vice versa.^[84]

Macros

Macros allow generation and transformation of Rust code to reduce repetition. Macros come in two forms, with *declarative macros* defined through `macro_rules!`, and *procedural macros*, which are defined in separate crates.^{[85][86]}

Because Rust lacks variadic functions or variadic templates, variadics are implemented using variadic macros, such as `println!` and `vec!`.^[87]

Declarative macros

A declarative macro (also called a "macro by example") is a macro, defined using the `macro_rules!` keyword, that uses pattern matching to determine its expansion.^{[88][89]} Below is an example that sums over all its arguments:

```
macro_rules! sum {
  ( $initial:expr $(, $expr:expr)* $(,)? ) => {
    $initial $(+ $expr)*
  }
}

fn main() {
  let x: i32 = sum!(1, 2, 3);
  println!("{x}"); // prints 6
}
```

Procedural macros

Procedural macros are Rust functions that run and modify the compiler's input token stream, before any other components are compiled. They are generally more flexible than declarative macros, but are more difficult to maintain due to their complexity.^{[90][91]}

Procedural macros come in three flavors:

- Function-like macros `custom!(...)`

- Derive macros `#[derive(CustomDerive)]`
- Attribute macros `#[custom_attribute]`

Interface with C and C++

Rust has a foreign function interface (FFI) that can be used both to call code written in languages such as C from Rust and to call Rust code from those languages. As of 2024, an external library called CXX exists for calling to or from C++.^[92] Rust and C differ in how they lay out structs in memory, so Rust structs may be given a `#[repr(C)]` attribute, forcing the same layout as the equivalent C struct.^[93]

See also

- C syntax
- C++ syntax
- Java syntax
- C# syntax

Notes

1. On Unix systems, this is often UTF-8 strings without an internal 0 byte. On Windows, this is UTF-16 strings without an internal 0 byte. Unlike these, `str` and `String` are always valid UTF-8 and can contain internal zeros.

References

1. Proven, Liam (2019-11-27). "Rebecca Rumbul named new CEO of The Rust Foundation" (https://www.theregister.com/2021/11/19/rust_foundation_ceo/). *The Register*. Archived (https://web.archive.org/web/20220714110957/https://www.theregister.com/2021/11/19/rust_foundation_ceo/) from the original on 2022-07-14. Retrieved 2022-07-14. "Both are curly bracket languages, with C-like syntax that makes them unintimidating for C programmers."
2. Vigliarolo, Brandon (2021-02-10). "The Rust programming language now has its own independent foundation" (<https://web.archive.org/web/20230320172900/https://www.techrepublic.com/article/the-rust-programming-language-now-has-its-own-independent-foundation/>). *TechRepublic*. Archived from the original (<https://www.techrepublic.com/article/the-rust-programming-language-now-has-its-own-independent-foundation/>) on 2023-03-20. Retrieved 2022-07-14.
3. Klabnik & Nichols 2019, p. 263.
4. Klabnik & Nichols 2019, pp. 5–6.
5. Klabnik & Nichols 2023, p. 32.
6. Klabnik & Nichols 2023, pp. 32–33.
7. Klabnik & Nichols 2023, pp. 49–50.
8. Klabnik & Nichols 2023, pp. 34–36.
9. Klabnik & Nichols 2023, pp. 6, 47, 53.
10. Klabnik & Nichols 2023, pp. 47–48.
11. Klabnik & Nichols 2023, pp. 50–53.

12. Klabnik & Nichols 2023, p. 56.
13. Klabnik & Nichols 2023, pp. 57–58.
14. Klabnik & Nichols 2023, pp. 54–56.
15. Klabnik & Nichols 2019, pp. 104–109.
16. Klabnik & Nichols 2019, pp. 24.
17. Klabnik & Nichols 2019, pp. 36–38.
18. Klabnik & Nichols 2023, pp. 36–38.
19. Klabnik & Nichols 2023, p. 502.
20. "Primitive type char" (<https://doc.rust-lang.org/std/primitive.char.html>). *doc.rust-lang.org*. Retrieved 7 September 2025.
21. "Glossary of Unicode Terms" (<https://www.unicode.org/glossary/>). *Unicode Consortium*. Archived (<https://web.archive.org/web/20180924092749/http://www.unicode.org/glossary/>) from the original on 2018-09-24. Retrieved 2024-07-30.
22. Klabnik & Nichols 2019, pp. 38–40.
23. Klabnik & Nichols 2023, pp. 40–42.
24. Klabnik & Nichols 2023, p. 42.
25. Klabnik & Nichols 2019, pp. 59–61.
26. Klabnik & Nichols 2019, pp. 63–68.
27. Klabnik & Nichols 2019, pp. 74–75.
28. Klabnik & Nichols 2023, pp. 71–72.
29. Balasubramanian, Abhiram; Baranowski, Marek S.; Burtsev, Anton; Panda, Aurojit; Rakamarić, Zvonimir; Ryzhyk, Leonid (2017-05-07). "System Programming in Rust" (<https://doi.org/10.1145/3102980.3103006>). *Proceedings of the 16th Workshop on Hot Topics in Operating Systems*. HotOS '17. New York, NY, US: Association for Computing Machinery. pp. 156–161. doi:10.1145/3102980.3103006 (<https://doi.org/10.1145%2F3102980.3103006>). ISBN 978-1-4503-5068-6. S2CID 24100599 (<https://api.semanticscholar.org/CorpusID:24100599>). Archived (<https://web.archive.org/web/20220611034046/https://dl.acm.org/doi/10.1145/3102980.3103006>) from the original on June 11, 2022. Retrieved June 1, 2022.
30. Klabnik & Nichols 2023, pp. 327–30.
31. "Lifetimes" (<https://doc.rust-lang.org/rust-by-example/scope/lifetime.html>). *Rust by Example*. Archived (<https://web.archive.org/web/20241116192422/https://doc.rust-lang.org/rust-by-example/scope/lifetime.html>) from the original on 2024-11-16. Retrieved 2024-10-29.
32. "Explicit annotation" (<https://doc.rust-lang.org/rust-by-example/scope/lifetime/explicit.html>). *Rust by Example*. Retrieved 2024-10-29.
33. Klabnik & Nichols 2019, p. 194.
34. Klabnik & Nichols 2019, pp. 75, 134.
35. Shamrell-Harrington, Nell (2022-04-15). "The Rust Borrow Checker – a Deep Dive" (<https://www.infoq.com/presentations/rust-borrow-checker/>). *InfoQ*. Archived (<https://web.archive.org/web/20220625140128/https://www.infoq.com/presentations/rust-borrow-checker/>) from the original on 2022-06-25. Retrieved 2022-06-25.
36. Klabnik & Nichols 2019, pp. 194–195.
37. Klabnik & Nichols 2023, pp. 208–12.
38. Klabnik & Nichols 2023, 4.2. References and Borrowing (<https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html>).
39. Pearce, David (17 April 2021). "A Lightweight Formalism for Reference Lifetimes and Borrowing in Rust" (<https://dl.acm.org/doi/10.1145/3443420>). *ACM Transactions on Programming Languages and Systems*. **43**: 1–73. doi:10.1145/3443420 (<https://doi.org/10.1145%2F3443420>). Archived (<https://web.archive.org/web/20240415053627/https://dl.acm.org/doi/10.1145/3443420>) from the original on 15 April 2024. Retrieved 11 December 2024.

40. Klabnik & Nichols 2019, p. 83.
41. Klabnik & Nichols 2019, p. 97.
42. Klabnik & Nichols 2019, pp. 98–101.
43. Klabnik & Nichols 2019, pp. 438–440.
44. Klabnik & Nichols 2019, pp. 93.
45. "OsStr in std::ffi" (<https://doc.rust-lang.org/beta/std/ffi/struct.OsStr.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230623184907/https://doc.rust-lang.org/beta/std/ffi/struct.OsStr.html>) from the original on 2023-06-23. Retrieved 2023-10-02.
46. "OsString in std::ffi" (<https://doc.rust-lang.org/beta/std/ffi/struct.OsString.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624103428/https://doc.rust-lang.org/beta/std/ffi/struct.OsString.html>) from the original on 2023-06-24. Retrieved 2023-10-02.
47. "Path in std::path" (<https://doc.rust-lang.org/beta/std/path/struct.Path.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624103428/https://doc.rust-lang.org/beta/std/path/struct.Path.html>) from the original on 2023-06-24. Retrieved 2023-10-02.
48. "PathBuf in std::path" (<https://doc.rust-lang.org/beta/std/path/struct.PathBuf.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230627034115/https://doc.rust-lang.org/beta/std/path/struct.PathBuf.html>) from the original on 2023-06-27. Retrieved 2023-10-02.
49. "std::boxed" (<https://doc.rust-lang.org/std/boxed/index.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230623184904/https://doc.rust-lang.org/std/boxed/index.html>) from the original on 2023-06-23. Retrieved 2023-06-23.
50. "Rc in std::rc" (<https://doc.rust-lang.org/beta/std/rc/struct.Rc.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624080501/https://doc.rust-lang.org/beta/std/rc/struct.Rc.html>) from the original on 2023-06-24. Retrieved 2023-06-24.
51. "Arc in std::sync" (<https://doc.rust-lang.org/beta/std/sync/struct.Arc.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624080503/https://doc.rust-lang.org/beta/std/sync/struct.Arc.html>) from the original on 2023-06-24. Retrieved 2023-06-24.
52. "Cell in std::cell" (<https://doc.rust-lang.org/beta/std/cell/struct.Cell.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624080506/https://doc.rust-lang.org/beta/std/cell/struct.Cell.html>) from the original on 2023-06-24. Retrieved 2023-06-24.
53. "Mutex in std::sync" (<https://doc.rust-lang.org/beta/std/sync/struct.Mutex.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624080502/https://doc.rust-lang.org/beta/std/sync/struct.Mutex.html>) from the original on 2023-06-24. Retrieved 2023-06-24.
54. "RwLock in std::sync" (<https://doc.rust-lang.org/beta/std/sync/struct.RwLock.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624080503/https://doc.rust-lang.org/beta/std/sync/struct.RwLock.html>) from the original on 2023-06-24. Retrieved 2023-06-24.
55. "Condvar in std::sync" (<https://doc.rust-lang.org/beta/std/sync/struct.Condvar.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624080504/https://doc.rust-lang.org/beta/std/sync/struct.Condvar.html>) from the original on 2023-06-24. Retrieved 2023-06-24.
56. "Duration in std::time" (<https://doc.rust-lang.org/beta/std/time/struct.Duration.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624080503/https://doc.rust-lang.org/beta/std/time/struct.Duration.html>) from the original on 2023-06-24. Retrieved 2023-06-24.

57. "HashMap in std::collections" (<https://doc.rust-lang.org/beta/std/collections/struct.HashMap.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624080505/https://doc.rust-lang.org/beta/std/collections/struct.HashMap.html>) from the original on 2023-06-24. Retrieved 2023-06-24.
58. "BTreeMap in std::collections" (<https://doc.rust-lang.org/beta/std/collections/struct.BTreeMap.html>). *The Rust Standard Library documentation*. Archived (<https://web.archive.org/web/20230624080503/https://doc.rust-lang.org/beta/std/collections/struct.BTreeMap.html>) from the original on 2023-06-24. Retrieved 2023-06-24.
59. McNamara 2021.
60. Klabnik & Nichols 2019, pp. 101–104.
61. "std::option" (<https://doc.rust-lang.org/std/option/index.html#representation>). *The Rust Standard Library documentation*. Retrieved 2023-11-12.
62. Klabnik & Nichols 2019, pp. 418–427.
63. "Casting" (<https://doc.rust-lang.org/rust-by-example/types/cast.html>). *Rust by Example*. Retrieved 2025-04-01.
64. Klabnik & Nichols 2019, pp. 171–172.
65. Klabnik & Nichols 2019, pp. 171–172, 205.
66. "Influences" (<https://doc.rust-lang.org/reference/influences.html>). *The Rust Reference*. Archived (<https://web.archive.org/web/20231126231034/https://doc.rust-lang.org/reference/influences.html>) from the original on November 26, 2023. Retrieved December 31, 2023.
67. Klabnik & Nichols 2019, pp. 181, 182.
68. Gjengset 2021, p. 25.
69. Klabnik & Nichols 2019, pp. 182–184.
70. Klabnik & Nichols 2019, pp. 281–283.
71. Klabnik & Nichols 2023, 18.2. Using Trait Objects That Allow for Values of Different Types (<https://doc.rust-lang.org/book/ch18-02-trait-objects.html>).
72. Klabnik & Nichols 2019, pp. 441–442.
73. Klabnik & Nichols 2019, pp. 379–380.
74. Rosenblatt, Seth (2013-04-03). "Samsung joins Mozilla's quest for Rust" (https://reviews.cnet.com/8301-3514_7-57577639/samsung-joins-mozillas-quest-for-rust/). CNET. Archived (https://web.archive.org/web/20130404142333/http://reviews.cnet.com/8301-3514_7-57577639/samsung-joins-mozillas-quest-for-rust/) from the original on 2013-04-04. Retrieved 2013-04-05.
75. Brown, Neil (2013-04-17). "A taste of Rust" (<https://lwn.net/Articles/547145/>). *LWN.net*. Archived (<https://web.archive.org/web/20130426010754/http://lwn.net/Articles/547145/>) from the original on 2013-04-26. Retrieved 2013-04-25.
76. "Races" (<https://doc.rust-lang.org/nomicon/races.html>). *The Rustonomicon*. Archived (<https://web.archive.org/web/20170710194643/https://doc.rust-lang.org/nomicon/races.html>) from the original on 2017-07-10. Retrieved 2017-07-03.
77. Vandervelden, Thibaut; De Smet, Ruben; Deac, Diana; Steenhaut, Kris; Braeken, An (7 September 2024). "Overview of Embedded Rust Operating Systems and Frameworks" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11398098>). *Sensors*. **24** (17): 5818. Bibcode:2024Senso..24.5818V (<https://ui.adsabs.harvard.edu/abs/2024Senso..24.5818V>). doi:10.3390/s24175818 (<https://doi.org/10.3390/s24175818>). PMC 11398098 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11398098>). PMID 39275729 (<https://pubmed.ncbi.nlm.nih.gov/39275729>).
78. "The Rust Language FAQ" (<https://web.archive.org/web/20150420104147/http://static.rust-lang.org/doc/master/complement-lang-faq.html>). *The Rust Programming Language*. 2015. Archived from the original (<http://static.rust-lang.org/doc/master/complement-lang-faq.html>) on 2015-04-20. Retrieved 2017-04-24.

79. McNamara 2021, p. 139, 376–379, 395.
80. "RAII" (<https://doc.rust-lang.org/rust-by-example/scope/raii.html>). *Rust by Example*. Archived (<https://web.archive.org/web/20190421131142/https://doc.rust-lang.org/rust-by-example/scope/raii.html>) from the original on 2019-04-21. Retrieved 2020-11-22.
81. "Abstraction without overhead: traits in Rust" (<https://blog.rust-lang.org/2015/05/11/traits.html>). *Rust Blog*. Archived (<https://web.archive.org/web/20210923101530/https://blog.rust-lang.org/2015/05/11/traits.html>) from the original on September 23, 2021. Retrieved October 19, 2021.
82. "Box, stack and heap" (<https://doc.rust-lang.org/stable/rust-by-example/std/box.html>). *Rust by Example*. Archived (<https://web.archive.org/web/20220531114141/https://doc.rust-lang.org/stable/rust-by-example/std/box.html>) from the original on 2022-05-31. Retrieved 2022-06-13.
83. Klabnik & Nichols 2019, pp. 70–75.
84. Klabnik & Nichols 2019, p. 323.
85. Klabnik & Nichols 2023, pp. 449–455.
86. Gjengset 2021, pp. 101–102.
87. The Rust Team (1 April 2026). "Variadic Interfaces" (<https://doc.rust-lang.org/rust-by-example/macros/variadics.html>). *rust-lang.org*. Rust by Example.
88. "Macros By Example" (<https://doc.rust-lang.org/reference/macros-by-example.html>). *The Rust Reference*. Archived (<https://web.archive.org/web/20230421052332/https://doc.rust-lang.org/reference/macros-by-example.html>) from the original on 2023-04-21. Retrieved 21 April 2023.
89. Klabnik & Nichols 2019, pp. 446–448.
90. "Procedural Macros" (<https://doc.rust-lang.org/reference/procedural-macros.html>). *The Rust Programming Language Reference*. Archived (<https://web.archive.org/web/20201107233444/https://doc.rust-lang.org/reference/procedural-macros.html>) from the original on 7 November 2020. Retrieved 23 Mar 2021.
91. Klabnik & Nichols 2019, pp. 449–455.
92. "Safe Interoperability between Rust and C++ with CXX" (<https://www.infoq.com/news/2020/12/cpp-rust-interop-cxx/>). *InfoQ*. 2020-12-06. Archived (<https://web.archive.org/web/20210122142035/https://www.infoq.com/news/2020/12/cpp-rust-interop-cxx/>) from the original on January 22, 2021. Retrieved 2021-01-03.
93. "Type layout" (<https://doc.rust-lang.org/reference/type-layout.html#the-c-representation>). *The Rust Reference*. Archived (<https://web.archive.org/web/20220716021840/https://doc.rust-lang.org/reference/type-layout.html#the-c-representation>) from the original on 2022-07-16. Retrieved 15 July 2022.

Sources

- Gjengset, Jon (2021). *Rust for Rustaceans* (1st ed.). No Starch Press. ISBN 9781718501850. OCLC 1277511986 (<https://search.worldcat.org/oclc/1277511986>).
- Klabnik, Steve; Nichols, Carol (2019-08-12). *The Rust Programming Language (Covers Rust 2018)* (<https://books.google.com/books?id=0Vv6DwAAQBAJ>). No Starch Press. ISBN 978-1-7185-0044-0.
- Klabnik, Steve; Nichols, Carol (2023). *The Rust programming language* (2nd ed.). No Starch Press. ISBN 978-1-7185-0310-6. OCLC 1363816350 (<https://search.worldcat.org/oclc/1363816350>).
- McNamara, Tim (2021). *Rust in Action*. Manning Publications. ISBN 978-1-6172-9455-6. OCLC 1153044639 (<https://search.worldcat.org/oclc/1153044639>).

